

Introduction to DOM

Until now, JavaScript was working only inside the browser console. For example, `console.log("Hello Students")` — the output appears in the console only. But can JavaScript change a heading, change button text, change an image, or change a webpage's color? **Yes — using the DOM.**

DOM stands for **Document Object Model**. It is a representation of your HTML page that JavaScript can understand and modify.

1

HTML

Browser creates DOM

2

DOM

JavaScript accesses DOM

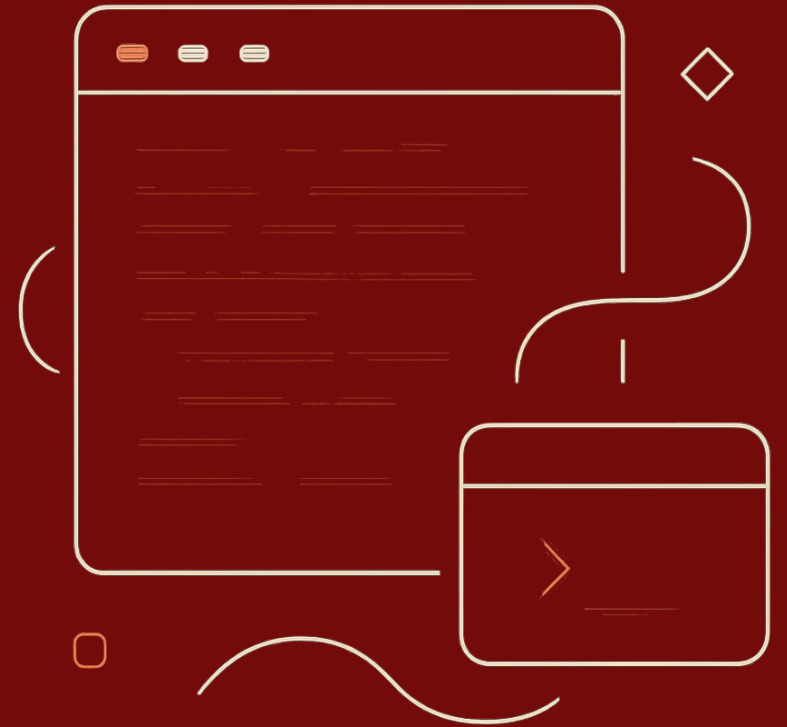
3

JavaScript

Webpage changes



Real World Example: Instagram — Like Button Click → Count Updates. JavaScript changes the DOM.



How DOM Works

When the browser loads HTML, it converts it into a **DOM Tree** — because JavaScript cannot understand HTML directly. The browser converts HTML into objects that JavaScript can access and modify.

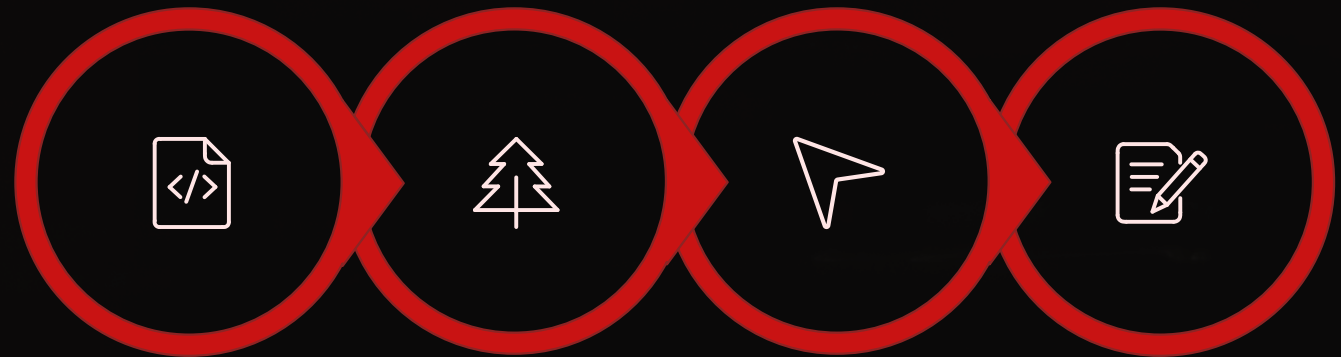
Example HTML

```
<h1>Welcome</h1>
<p>Hello Students</p>
```

DOM Tree Structure

```
Document
├── h1
│   └── "Welcome"
└── p
    └── "Hello Students"
```

Visual Flow



**HTML
Loads**

Build DOM

**JS
Accesses**

**JS
Modifies**

Open browser DevTools and check the **Elements Tab** — this is the actual DOM.



Selecting Elements

Before changing anything, JavaScript must **locate the element**. Just like before calling a friend you need their phone number — before changing HTML, you need an element reference.

① Select Element

Locate the target in the DOM

② Store Element

Save reference in a variable

③ Modify Element

Change content, style, or behavior

```
let heading = document.getElementById("title");  
// Now JavaScript knows which element to work with
```

📘 **Real World Example:** Select Login Button → Add Click Event

getElementById()

Concept

Every id is **unique**. Only one element should have a particular id. The browser searches, finds the matching element, and returns it.

```
let heading = document.getElementById("title");  
heading.innerText = "JavaScript DOM";
```

Before

Welcome Students

Real World Usage



Profile Name Update



Cart Count Update



Success Messages

After

JavaScript DOM

querySelector()

The **modern way** of selecting elements. Instead of learning multiple methods, one method can select anything — by id, class, or tag. Most modern JavaScript projects use `querySelector()`.

Select by ID

```
document.querySelector("#title");
```

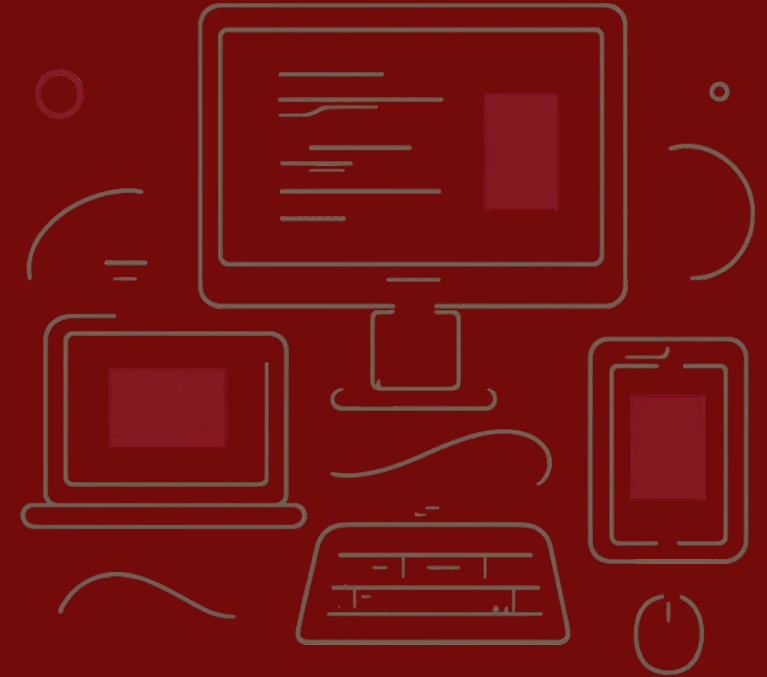
Select by Class

```
document.querySelector(".card");
```

Select by Tag

```
document.querySelector("h1");
```

⚠ **Important Rule:** `querySelector()` returns **only the first match** found in the DOM.



querySelectorAll()

The Problem

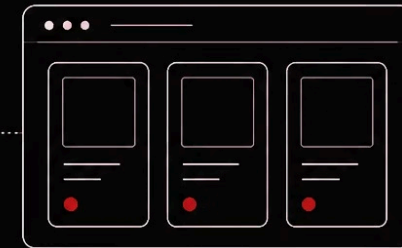
What if **multiple elements** exist? `querySelector()` only returns the first paragraph. To access all of them, use `querySelectorAll()`.

```
let items = document.querySelectorAll("p");  
console.log(items[0]);  
console.log(items[1]);
```

This returns a **collection of elements** — Paragraph 1, Paragraph 2, Paragraph 3 — all stored together.

Product Cards

(select all product cards on a shop page)



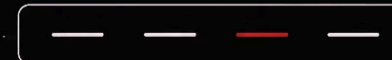
Student Lists

(select all student rows in a table)



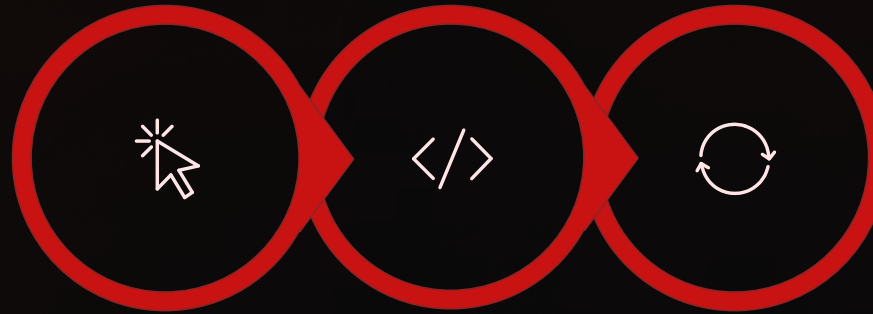
Menu Items

(select all navigation links)



Changing Content Using innerText

After selecting an element, we can **modify its content** using `innerText`. This is how dynamic text updates happen on real websites.



**Select
Element**

**Set
innerText**

**Content
Updates**

Select the element → use `innerText` → content changes on the webpage instantly.

Example Code

```
let heading = document.getElementById("title");  
heading.innerText = "Welcome to JavaScript";
```

Real World Usage

🔑 **Login Successful**

📦 **Order Confirmed**

💳 **Payment Completed**

✓ **Teaching Activity:** Change the heading to a student's own name using `innerText`.



Mini Project: Dynamic Greeting Webpage

Create an **interaction between user and webpage**. When the button is clicked, JavaScript runs and updates the DOM with a personalized greeting.

HTML

```
<h1 id="title">Welcome</h1>
<button id="btn">Click Me</button>
<p id="message"></p>
```

JavaScript

```
let button = document.getElementById("btn");
let message =
document.getElementById("message");

button.onclick = function(){
  message.innerText = "Good Morning Students";
}
```

Project Flow

01

Button Click

User interacts with the page

02

Event Triggered

onclick handler fires

03

JavaScript Runs

Function executes

04

DOM Updated

innerText changes on screen



Used in: Welcome Messages, Notifications, Alerts

Complete Practical Understanding

Let's bring everything together in a complete step-by-step example that demonstrates the full DOM workflow from HTML to interactivity.

1 Create HTML

```
<h1 id="heading">Hello</h1>
<button id="changeBtn">Change Text</button>
```

2 Select Elements

```
let heading = document.getElementById("heading");
let button = document.getElementById("changeBtn");
```

3 Add Functionality

```
button.onclick = function(){
  heading.innerText = "JavaScript is Awesome";
}
```

DOM Connection

Select elements and expose them to scripts.

JavaScript Logic

Add event handlers and state changes.

HTML Structure

Define headings, buttons, and elements.

User Interaction

Click or input to trigger dynamic updates.

Students learn the complete chain: **HTML → DOM → JavaScript → Interaction**

Summary, Homework & Outcome

Key Concepts Learned

DOM

Bridge between HTML and JavaScript

getElementById()

Select an element using its unique ID

querySelector()

Select the first matching element

querySelectorAll()

Select multiple elements at once

innerText

Change content dynamically

Homework

Build a **Student Introduction Page** with the following requirements:

- A heading element
- A paragraph element
- A button element

When the button is clicked: **change the heading** and **change the paragraph** text.

Final Outcome

- Understand DOM
- Select HTML Elements
- Modify Content
- Create Interactivity
- Build Dynamic Webpages